Steganography with Language Models

Machine Learning and Security - Project Report
Prof. Dr. Konrad Rieck
Technical University of Berlin

Alexander Malinowski
Lynn Janzen

July 2024

Abstract

We present methods to hide a given secret message within a natural seeming cover text, restore a message within such a text, and detect whether a message has been stored, all using large language models (LLMs). We encode a secret by encoding it byte by byte into the tokens of a text as represented by an LLM according to their relative probability, and restore the secret via the same mechanism in reverse. We detect whether a text is LLM-generated by checking if it lies near the top of the probability distribution as indicated by an LLM.

1 Introduction

Steganography is the practice of concealing a message, image, or file within another media to avoid detection. Unlike cryptography, which makes data unreadable without a key, steganography hides the existence of the message itself. An advantage of steganography over cryptography is that it does not attract attention. While encrypted data signals the presence of sensitive information, data hidden via steganography blends in with its context, making it less likely to be discovered, if done correctly.

The aim of the course "Steganography with Language Models" in the summer semester 2024 was to develop a method to hide a secret message consisting of a Base64-encoded string in a message feed, as well as to develop approaches to detect the manipulation of given texts. Three groups of two to three students competed against each other and tried to expose the generated news feeds of the other groups against texts without a hidden message. The provided news feeds contained of 30 articles, each with an average length of 500 words. In the following, we will explain the approaches for our Hider and Detector modules, explain the implementations, and present and discuss test results. For all three modules, hider, restorer and detector, we relied on Llama2 (version llama-2-7b)[TMS+23] as provided by the lecturers.

2 Hider & Restorer

2.1 Concept

In the context of text steganography, *perceptual imperceptibility* refers to the ability to embed hidden information within text without altering its readability to human readers. This ensures that the stego text looks and feels natural, preventing suspicion. However, optimizing only for perceptual imperceptibility is insufficient because advanced detection techniques can analyze patterns and irregularities that might not be noticeable to humans but are detectable through statistical analysis. *Statistical imperceptibility*, therefore, involves ensuring that the frequency and distribution of characters, words, or phrases in the stego text align with those of the cover texts to avoid detection by statistical methods. Additionally, *cognitive imperceptibility* is based on maintaining the semantic coherence and logical consistency of the text ([YXZ+21]).
Our approach was intended to cover perceptual and statistical imperceptibility criteria and since we employed a generative approach we expected cognitive imperceptibility to follow from these. We based our approach on LLsM by Wang et al.[WSZ+24]. In their work, the authors present an idea to improve the quality of existing stego schemes by utilizing the model scores of the candidate tokens to encode the secret and also by dynamically determining a padding interval between coding tokens in the text in order to control the embedding rate. We made use of these two concepts

in our implementation and furthermore experimented with different pre-processing schemes of the secret in order to improve perceptual imperceptibility further.

2.2 Implementation

Encoding in our approach took place on token level, by prompting the model with the first 64 tokens of the original news article. Using the llama-cpp-python library [1] to access the models internals, we generated a list of the next candidate token and their scores under the model. After sorting the list by the scores we could use the secret bytes as indices in the list of candidate tokens and select the coding token accordingly (Algorithm 7). With this basic approach the value space of each byte and thus possible index would range from 0 to $2^8 - 1$. The restorer works exactly inversely (Algorithm 6).

Algorithm 1 Hider Algorithm

Require: Tokenized sequence x, Secret y as byte array
Ensure: Stego text
1: Skip first 64 tokens
2: Convert secret into bytes
3: **for** each byte k in y **do**
4: Generate the list of the next candidate tokens and their probabilities under the model
5: Sort list by the probabilities
6: Choose the kth token
7: Skip the next n tokens

Algorithm 2 Restorer Algorithm

Require: Sequence x
Ensure: Secret y
1: **for** each number k in the value space (0 - 256) **do**
2: Extract exactly that candidate token whose rank in the sorted list of candidate tokens equals k
3: **if** it equals the actual next token in the stego text **then**
4: Append k to the secret
5: **break**
6: Determine the amount of tokens to skip, based on the logit probability of the selected token

2.3 Pre-evaluation

If tokens that code for the hidden information (called *coding tokens* in the following) follow each other without padding, not only is the secret embedded in one single line, which might ease detection, but also the quality of the stego text decreases drastically, as errors due to the selection of low quality tokens propagate and very quickly pile up. Furthermore the resulting text is very short and thus suspicious. Thus, to distribute the coding tokens over the entire text we started out by utilizing the model score of the coding token rounded to the nearest integer as the interval size to the next coding token. This gave us a dynamic interval which would cover the coding tokens location better and should better simulate the distribution of natural texts[WSZ+24].

This approach returned texts that were markedly shorter than the original news articles, so we decided to adjust the interval length so that the target length of about 500 words would be met. Another idea was to empirically determine a suitable factor for the logits used as interval length but due to the very long execution times of our algorithm we decided to instead precompute a fixed interval length based on the chunk size of the secrect, the target length of the output and an approximate tokens per word ratio of 0.75, that would yield the desired text length on average. This of course came at the cost of making the coding token position in the text more predictable.

When testing this setup we encountered that the resulting stego text lacked perceptual imperceptibility since some of the coding tokens had low model scores due to the large value space of the indices. Since we fed the context window of the model with more and more low probability tokens the predictions of the model also worsened. Not only did the text contain spelling and grammatical errors that made it suspicious, but also some of the locations of the secret tokens were immediately

[1] https://github.com/abetlen/llama-cpp-python

visible (see Figure 1(a)). A detector with an integrated spell checker would be able to detect these locations as well as an english-speaking human inspector.

To improve the quality of token choices we experimented with splitting the secret into chunks of different size in order to reduce the value space, and assembling the secret back together after the decoding step. We tried chunks of four and chunks of two bits and were expecting the readability to improve for four over eight bits but also even more for two over four bits. At the same time, to reach the target word count, we precomputed the amount of tokens in between code tokens.

When comparing the perceptual qualities of the three methods on some examples we were able to see an increase in readability for the 4 bit method with precomputed interval size of 17 over our initial version with 8 bits and a varying length between 2-8 tokens (Figure 2(a)). Since the 2 bit method did not provide an improvement to the readability that would be proportional to its increased computing time (Figure 2(b)), we decided to omit it. Notably the increase of the interval length in the 8 bit method resulting from our shift from using the token scores as base for the interval to a much longer precomputed length also increased its perceptual quality (Figure 1(b)). Since the passages within the padding interval are generated always using the candidate token with the highest score, longer padding intervals can probably better compensate for more unsuitable token choices in the 8 bit method and thus possible errors are less likely to propagate.

...Jewish National Fund rescue efforts – has been viewed **also** by Israeli police.
The video **posts** were accompanied by a warning that the militiakiller was still at large.
"We are looking for this terrorist," said **Po**liceman Yossi Dahan, who **p**ledged to "do everything in our power lest he hurt anyone else."
The attack **injured** 16 people and killed two others – **had**assah medical center sposted on Sunday that the death toll was now three.
The video shows a man **beating** a woman with his rifle butt **there**after shooting her at point-blank range, then **taking** her purse.
"I'm going to kill yourself," he says in Arabic as she lies bleidding on the ground. "You are dead."
He **added**: "I will kill you and your family." **According** to the video, he also shot a man who was trying to **ph**ysically intervene.
The attacker then **appears** to have looted the belongings of other victims before **ro**bbing a car at gunpoint.
"**Three** people were killed in the attack," said Israeleri police spokesperson Micky **El**ran, adding that 16 others were injured Including two seriously.
"The terrorist is **acting** alone and we are looking for him," he Haddan said. **Else**where on Sunday, Israel's military said **to** have killed a PalestinHamas militAn in Gaz.
The attack **es**calated tensions between Israel and Ham

(a)

-listed restaurants, is the Hong Kong Tatler Dining Club (HK **Ts**D).
The HK TsD is a private dining club that has been racking up accolades since it was founded in 2013. **Early** on, it won the Best Newcomer Award at the 2014 **AG**FG Awards and the Best Dining Club Award at the 2015 **CNN** Travel Awards.
The HK TsD is a private dining club that **extends** invitations to its members for exclusive events with top chefs, winemakers and **guests** from around the world. It also organizes bespoke tours of Hong Kong **Ch**inatown's food markets and restaurants, as well as wine-**ded**icated trips to Bordeaux and Burgundy in France.
The lion's share of its events are held at the HK TsD's **temporary** home on the 28th floor of the Hong Kong Club Building (1 Jackson **Ro**ads). The club is a private space that can be rented out for exclusive **du**ding experiences, and it has hosted everything from intimate wine tastings to large **tables**ide banquets..

(b)

Figure 1: Sample passages of 8 bit stego text. Bold tokens mark the location of the secret. Left: Stego text resulting from our first version. Coding tokens are distributed over the text at varying interval size. Tokens in bold mark coding tokens, red tokens denote suspicious locations were the coding token produced spelling or grammatical errors. Right: Stego text resulting from 8 bit secret size and preomputed interval length of 17. Prompt: "*Right at the heart of its culinary world, with connections to at least half of its h...*".

..anging-out spots, is the city **center**'s Central district.
The area **features** some of Hong Kong's most expensive, high-end restaurants – but also **do**zens of casual eateries **and** street food stalls that are just as **essential** to the city's dining scene **and** its residents' daily lives.
In **our** latest video, we take a look at **the** Central district's culinary landscape, **speaking** with chefs, restaurant owners and **loc**als about what makes this part of town **essential** for Hong Kong's food culture. **And** we also sample some of the best **dough**nuts in the city – from a 10-year-old stall selling them out of a cart to a **brand** new shop that has already become a local **institution**.
The video is part of our **C**ulinary Journeys series, **featuring** food and drinks from around the world – including Hong Kong's best **dumsums**..

(a)

..allowed restaurants, **lies** the Hong Kong **W**ine & **DINE** Group.
In 20 **years**, the company, founded by CE**Os** David Chu **&** Michael Chau **has** grown from a **single** restaurant in Hong-Kong'**S** Central District into **the** largest privately-owned restaurant group **with** over 120 restaurants, bars and **ca**fes across Asia.
Their **flagship** restaurant, **the** Michelin star, three-star **restaurant** Bo Inno**vations**, is one **such** example of their **unparalleled** **expertise** in the **industry**.
The restaurant, which has been **named** as one of **Hong** Kong's **most** expensive restaurants, serves a **9**-course **tribute** to the fin**est** ingredients **in** the world. **The** menu changes every **two** months and features **d**ishes like "**S**weet Corn with **Foie** Gras **&** Caviar" **&** "Sea **Bass** with Scal**loped** Potatoes". **It** is also one **of** the most expensive **restaurants** in Hong-Kong, **costing** up to **HK$1,000** **per** person..

(b)

Figure 2: Sample passages of 4 and 2 bit stego text. Bold tokens mark the location of the secret. Prompt: "*Right at the heart of its culinary world, with connections to at least half of its h...*". Left: Stego text resulting from 4 bit secret size and precomputed interval length of 8. Right: 2 bit secret and precomputed interval length of 3 tokens.

2.4 Issues

Since execution time for the 4 bit method was significantly longer that 8 bit (see Table 1), we initially decided to stick to the latter and use a long interval. During restoring we discovered that the secret could not reliably be recovered from longer generated text. Upon investigation we found that the cause for this issue was based on small divergence between tokenization in the encoding and decoding stage. Figure 3 shows an extract from our logs, the encoding stage being depicted on the left, decoding on the right side. As visible in the reddish marked line the token that was selected by the encoder to code for the specific byte was not correctly mapped by the tokenizer in the decoder which causes the decoding process to run out of the scheme and results in a failure to restore the rest of the secret.

This problem occurred in most test cases where the model was used to generate texts that match the average length of 500 words, but did not occur with smaller interval lengths, resulting in shorter texts. A possible cause to this problem could lie in rounding errors during generation

Algorithm 3 The original DetectGPT algorithm[MLK+23]

Input: passage x, source model p_θ, perturbation function q, number of perturbations k, decision threshold ϵ

Output: True – probably machine-generated, False – probably human-written.

function DETECTGPT(x, p_θ, q, k, ϵ)
  $\tilde{x}_i \sim q(\cdot \mid x)$, $i \in [1..k]$ ▷ mask spans, sample replacements
  $\tilde{\mu} \leftarrow \frac{1}{k} \sum_i \log p_\theta(\tilde{x}_i)$ ▷ approximate expectation in Eq. 1
  $\hat{\mathbf{d}}_x \leftarrow \log p_\theta(x) - \tilde{\mu}$ ▷ estimate $\mathbf{d}(x, p_\theta, q)$
  $\tilde{\sigma}_x^2 \leftarrow \frac{1}{k-1} \sum_i (\log p_\theta(\tilde{x}_i) - \tilde{\mu})^2$ ▷ variance for normalization
  if $\frac{\hat{\mathbf{d}}_x}{\sqrt{\tilde{\sigma}_x}} > \epsilon$ **then**
    return `true` ▷ probably model sample
  else
    return `false` ▷ probably not model sample

that appear with a greater probability for longer inputs and might lead to slightly different token scores. Although we could not find the definite cause to this phenomena, other users do report non-determinism issues related to floating point conversion[2], which could be a hint. With our final configuration of a 4 bit secret with a static skiplength of 2 tokens the secret could be restored fully.

Secret Type	Skip Length (tokens)	Execution Time (mins)
8 bit	17	9
4 bit	8	16
4 bit	2	9
2 bit	3	28

Table 1: Execution times

```
..
encoding secret 186 as 29899 which is b'-'
skipping the next 2 tokens
encoding secret 249 as 17173 which is b'
wealth'
skipping the next 2 tokens
encoding secret 242 as 11464 which is
b'ationale'
skipping the next 2 tokens
encoding secret 179 as 266 which is b' th'
skipping the next 2 tokens
..
```

```
..
Decoding byte of 29899 which is b'-'
Skip length: 2
Decoding byte of 17173 which is b' wealth'
Skip length: 2
Decoding byte of 2633 which is b'uration'
Skip length: 2
Decoding byte of 29892 which is b','
Skip length: 2
..
```

Figure 3: Extract from our logs to illustrate the tokenization differences between encoding and decoding that occur for longer generated texts. Log taken from 2 bit secret with 2 token intervall length.

3 Detector

3.1 Concept

For the purpose of this seminar that posed a relatively constrained environment in which it was clear that all stego texts of the different groups have to contain an amount of machine generated text, it was viable to reduce the problem of stego text detection to the one of discriminating machine generated text from human written text.

Our detector is split into two parts: a statistical evaluation of the length of the texts and an LLM-based evaluation to check whether the text was generated by a language model. We then

[2] see for example https://github.com/vllm-project/vllm/issues/966 and https://discourse.julialang.org/t/llama2-7b-difference-in-inference-when-between-float16-and-float32/103826

Algorithm 4 The improved Fast-DetectGPT algorithm[BZT+24]

Input: passage x, sampling model q_φ, scoring model p_θ, and decision threshold ϵ
Output: True – probably machine-generated, False – probably human-written.

function FastDetectGPT(x, q_φ, p_θ)
  $\tilde{x}_i \sim q_\varphi(\tilde{x}|x), i \in [1..N]$ ▷ Conditional sampling
  $\tilde{\mu} \leftarrow \frac{1}{N}\sum_i \log p_\theta(\tilde{x}_i|x)$ ▷ Estimate the mean
  $\tilde{\sigma}^2 \leftarrow \frac{1}{N-1}\sum_i (\log p_\theta(\tilde{x}_i|x) - \tilde{\mu})^2$ ▷ Estimate the variance
  $\hat{\mathbf{d}}_x \leftarrow (\log p_\theta(x) - \tilde{\mu})/\tilde{\sigma}$ ▷ Estimate conditional probability curvature
  return $\hat{\mathbf{d}}_x > \epsilon$

combine those by adding them together in the form of logits (i.e. $\ln \frac{p}{1-p}$), which have the property that 50% is equivalent to a logit value of 0, making it easy to weigh different approaches differently without changing their decision. We then label texts as LLM-generated when their probability in logits is > 0. In order to have a prior p to add to, we used the knowledge that the score of the detector is computed as the F_1-score (Equation 1).

$$F_1 = \frac{2tp}{2tp + fp + fn} \tag{1}$$

When we upload a detector that just always says "yes", fn becomes 0 and fp becomes $1 - tp$. We can then solve for the ratio of true positives (Equation 2) to get a perfect prior.

$$tp = -\frac{1}{(1 - \frac{2}{F_1})} \tag{2}$$

For the statistical approach, we assume that the lengths of the texts follow a normal distribution (Equation 3).

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} \tag{3}$$

From the provided example feeds we deduced a mean of $\mu = 450$ words and a standard deviation of $\sigma \approx 50$. We can then use the normal error function (Equation 4) to compute the probability that a given length occurs.

$$\mathrm{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^{\sqrt{2}\frac{x-\mu}{\sigma}} e^{-t^2}\, dt \tag{4}$$

The LLM-based detector was initially based on DetectGPT[MLK+23]. DetectGPT works by taking in the text, and at randomly sampled points in the text perturbing it by exchanging words with new ones sampled from the model, then evaluating the probabilities of the original text and the perturbed one under the model (see Algorithm 3). The idea being that a text that was previously generated by this model will have a very high probability under the model, so swapping out words will on average lower the probability of the text. By contrast, a text that was not generated by the model will see an increase in probability on average when swapping out words, as the model is free to choose the words it assigns the highest probability to. More formally, given a text passage x, an LLM will try to generate a continuation $\tilde{x}$ that maximizes the probability $p(\tilde{x}|x)$, whereas any individual human does not have this as a goal when writing. Therefore, the probability $p(y|x)$ of the token y actually encountered in the text as computed by the model will on average be at or near the peak of the probability distribution for texts that are LLM-generated, but lower for human-written texts. Substituting tokens that are at the top of the probability curve will then lead to a lower probability of the perturbed text, while generating alternative tokens for tokens that are not of high probability will raise their probability.

While this approach worked, it was very slow, taking upwards of two hours per feed. To remedy this, we switched to Fast-DetectGPT[BZT+24]. Here, instead comparing the text with a perturbed version of it, we compare the words with the probability distribution of the model. To do so, we generate the models scores for the next token at chosen points in the text and sample 10000 tokens and their corresponding scores from this list. Next we compute the mean μ and standard deviation σ of those samples. We can then take the probability p of the token that is actually present in the text and compute $\frac{p-\mu}{\sigma}$ to locate it within the distribution (see Algorithm 4). The main idea here is very similar as the one for DetectGPT above. A token generated by a model as a continuation of a text will on average be very close to the mean of the probability distribution that generates the token. While the models are trained on human texts and do try to mimic the probability distribution of human-written texts, any individual human is unlikely to write exactly

as the average human does, and therefore the probabilities of tokens in human-written texts will on average stray farther from the probability distribution than the ones from LLM-generated ones. In the Fast-DetectGPT paper itself, this evaluation is done for every point in the text. However, because of constraints on time and processing power, our method only evaluated the text at at first 5 and later on 50 uniformly distributed points within the texts.

Both of these methods assume that if the text is LLM-generated, then it is generated by the same model that we use in our detector to generate our probabilities. This is of course not a realistic assumption to make in practice, as there are a wide variety of models available, with new ones being published on an almost daily basis. However, as we will see below in Section 4, our detector also works reasonably well on texts generated by different models, likely because almost all large language models share a significant part of their training data.

3.2 Implementation

To implement Fast-DetectGPT we used Llama 2 7b[TMS+23] as our base model, interfacing with it using llama-cpp-python. This proved to be slightly problematic, as the API was not made with this use-case in mind. For example, accessing and modifying the logits was only possible in a very roundabout way. There is no method that directly returns them, instead it is mandatory to implement a `LogitsProcessorList`, which is a list of objects of type

`llama_cpp.LogitsProcessor = Callable[[npt.NDArray[np.intc], npt.NDArray[np.single]], npt.NDArray[np.single]]`.

This is a method that will then get called before the actual sampling of the next token, where the second argument and the returned value is the list of logits. This was unfortunately poorly documented.

Another problem was that sampling a token from the model automatically adds it to the current context. This is problematic when wanting to sample at a set point 10000 times. Thankfully, the Fast-DetectGPT paper[BZT+24] itself directly pointed out the solution to this problem, which is to extract the logit probabilities from the model and then use

`torch.distributions.categorical.Categorical(logits=lprobs).sample([10000])`

to sample from the distribution directly.

A different problem was that the computed standard deviation was sometimes very small. This then lead to overflows when dividing by the standard deviation. The solution was to clamp the value to be a minimum of 0.0001, which did not seem to negatively impact performance and provided the needed stability.

4 Evaluation

For evaluation of our hider and detector we investigated 4 settings. Firstly we internally tested the performance of our detector against stego texts generated by our detector, secondly we externally verified the detector's capabilites to detect machine generated text that was created by another language model than Llama2. Lastly, we compared our detectors performance in both conditions against the author's implementation of FastDetectGPT[3] which we altered slightly to use Llama2 as scoring model for our tests. This implementation followed a slightly different approach by not retrieving model scores from a smaller amount of sampling points but instead evaluating the whole input passage, thus assumingly yielding more reliable results.

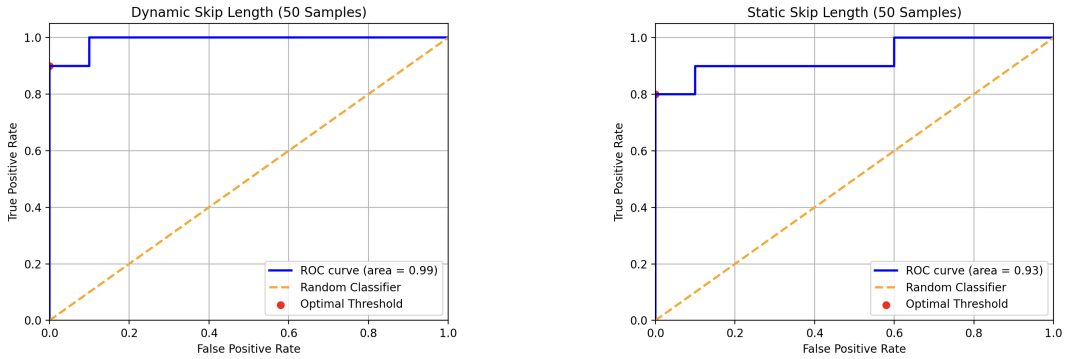
4.1 Internal evaluation

Following the experimental approach outlined in the paper describing FastDetectGPT ([BZT+24]) we constructed two bodies of text per condition, consisting of 10 news articles each, were the first body was serving as the machine generated positive samples and the second as the human written negative samples.

For generating the machine generated corpus, we prompted the model with the first 64 tokens of a corresponding news article (provided by the lecturers) using our 4 bit approach to hide the secret. To evaluate the performance of our hider, we used the precomputed (dynamic) interval length of 8 tokens and compared it against the final configuration of 4 bit, fixed 2 token interval length (static) and an appended filler text to match the 500 words target length. The original news articles were used as negative samples in both conditions. Detection accuracy was measured by computing the area under the receiver operating curve (AUROC) to gain comparability with FastDetectGPT.

[3] https://github.com/baoguangsheng/fast-detect-gpt/tree/main

We naturally expected our detector, which is essentially a binary classifier, to yield a "better" detection performance on the dynamic length condition, since the entire passage following the first 64 tokens was machine generated, whereas the detector should be fooled by the static length condition, since the machine generated passage was shorter and embedded in human written parts. Figure 4 depicts the RO curves of the different experimental conditions. Descriptively, the detector scores at higher accuracy in the dynamic length condition, were the whole passage is machine generated (AUROC = 0.99). For the static condition the result is slightly decreased (AUROC = 0.93) which is in line with our expectation but due to our small sample size of 20 texts per condition cannot be interpreted robustly.

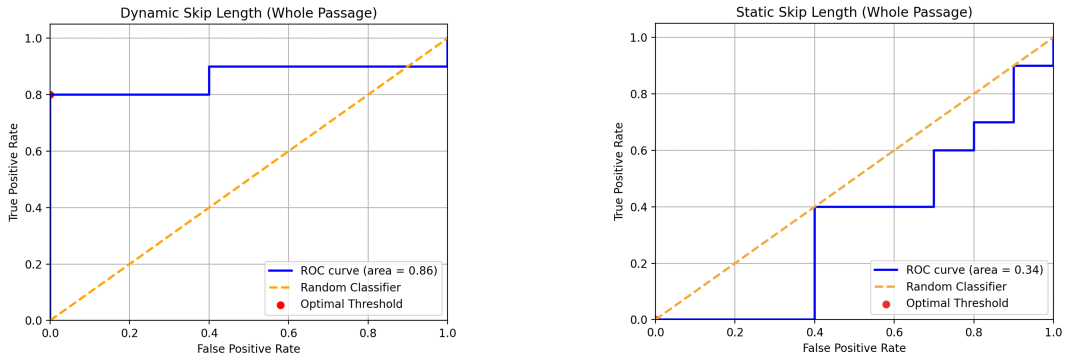


(a) (b)

Figure 4: ROC curves for dynamic and static skip length when evaluating at 50 points in the texts.

4.1.1 Comparison with FastDetectGPT

The same text samples were used for the comparison with the original FastDetectGPT algorithm. Likewise we expected the dynamic length condition to be detected with higher accuracy over the static condition.
In Figure 5 it is visible that FastDetectGPT scores much more in line with our expectation, classifying the dynamic condition with high accuracy (AUROC = 0.86) but failing to detect the machine generated parts in the static one (AUROC = 0.34). This points to the conclusion that a sufficient amount of sampling points per text are crucial for reliable detection.



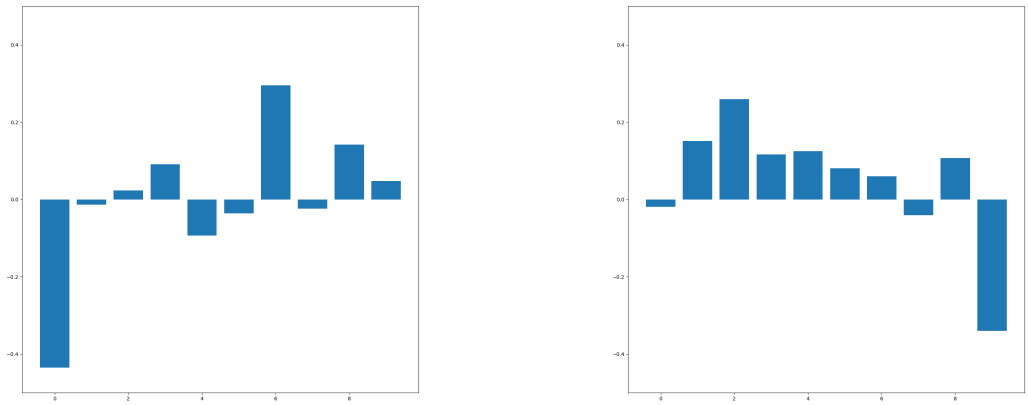
(a) (b)

Figure 5: The left figure shows the ROC curve for 4 bit stego texts with dynamic skip length (precomputed) to match the target length of 500 words. The right graph corresponds to the version with a static skip length of 2 tokens and an appended filler text. The detector samples on the whole input passage and the comparison baseline are human written articles taken from the provided news feeds.

4.2 External Evaluation

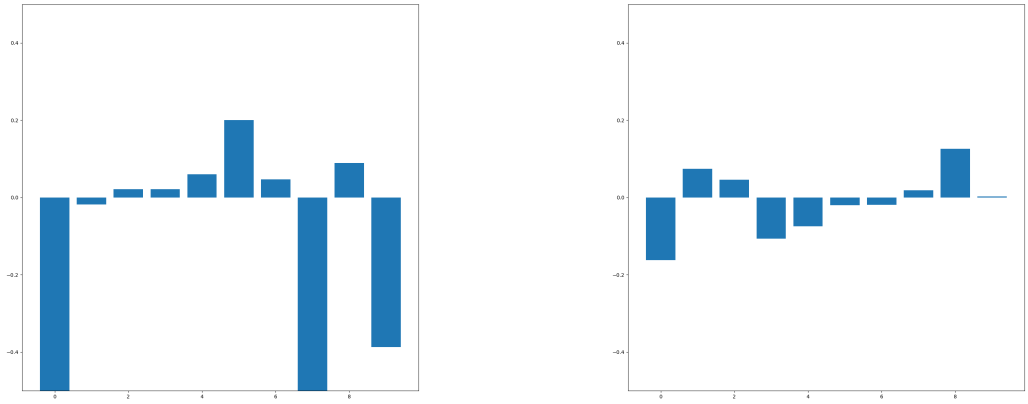
To evaluate the detector on LLM-generated content in general, we generated ten news articles using the model Dolphin Mixtral 2x7b with the prompt "Write a news article in the style of the

BBC". Ten BBC news articles from the internet served as negative samples.



(a) (b)

Figure 6: Logit scores of Dolphin Mixtral 2x7b generated news articles being LLM-generated. A value of 0 is equivalent to a probability of 50%. Bars going up indicate that the model thinks the text is LLM-generated. Left are the probabilities when evaluating at 5 points, right the probabilities when evaluating at 50 points.

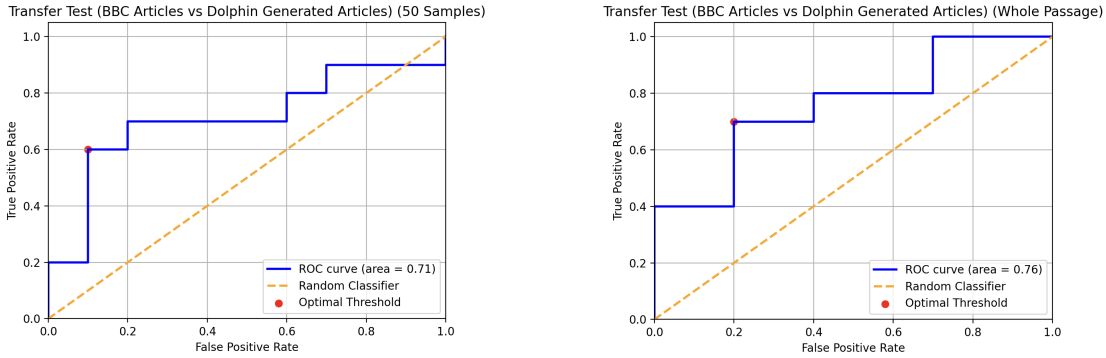


(a) (b)

Figure 7: Logit scores of human written news articles being LLM-generated when evaluating at 5 (left) or 50 (right) points in the text

The logit scores of the machine generated and the human written articles are plotted in Figure 7 and 6 respectively, showing a sampling point number of five on the left and 50 on the right. In the left subgraph of Figure 7 it can be seen that the probabilities are all relatively close to 50% and only five of ten texts are correctly flagged as machine generated, indicating relatively high uncertainty in the classification process. When we increase the number of sampling point in the text from five to 50 points (Figure 7 right), the model can correctly identify 7 out of 10 texts as being LLM-generated.

By the nature of the detector's implementation, it should perform better on text that was generated by the same model as was used in detection, using the same underlying logit distributions. When plotting these results as a receiver operating characteristic (ROC) curve (Figure 8(a)) we can see that our model performs significantly better than chance at distinguishing the human written texts from the dolphin 2x7b generated ones, despite it being a different model than the one we use to score, and at the same time does not reach the same performance as the stego texts (AUROC = 0.71 vs 0.99), which is in line with the expectation.



(a) (b)

Figure 8: In the transfer test machine generated texts (Dolphin) are compared against a range of human written BBC news articles. The left figure shows the detection based on 50 samples, the right corresponds to evaluating the whole passage.

4.2.1 Comparison with FastDetectGPT

When feeding the same text bodies into FastDetectGPT we yield a slightly better performance as compared to our detector (Figure 8(b), AUROC = 0.71 vs. 0.76), again underpinning the notion that many samples are needed for reliable detection.

5 Discussion

In our results presented above we find conflicting results regarding the comparison of our detector and the performance of FastDetectGPT when classifying our generated stego texts. The performance of our detector being seemingly better on the dynamic length condition that FastDetectGPT despite less sample points, in conjunction with our detector scoring very high on texts that contain a large portion of human written text suggests we got lucky with the points being sampled being of above average usefulness and is likely a result of our relatively low sample size of twenty texts. As mentioned above it is not only a question of the number of articles in the text body but also crucial to sample at more points in the text to yield good detection result, as visible by FastDetectGPT's evaluation being more in line with out expectations.
Even though we were able to tune the hider's performance to satisfy perceptual imperceptibility requirements, despite the tokenizer issue, it is worth mentioning that in some of our stego texts the model wandered off topic during text generation, suddenly switching to another topic or writing style (eg. from news article to blog post). While this might not affect LLM-based detection like DetectGPT or FastDetectGPT, it surely negatively impacts cognitive imperceptibility. [YXZ+21]
Some additional factors that could have impacted our results are worth mentioning: We constrained our evaluation to news articles, since that is what the task required, and the performance on other types of text might be different. Importantly, we also assumed that the articles taken from the BBC were not LLM-generated. Worst of all, our sample size is very small. Unfortunately, evaluating our model takes a rather long time, and we simply do not possess the hardware necessary to do large-scale evaluations on something like the XSum[NCL18] dataset.
Someone trying to fool our detector has a few options. Using a different model than Llama 2 to generate their texts is a start if the attacker knows that we are using Llama 2, though as we have shown above our detector is still capable of detecting texts by other models, likely because most large language models share a significant volume of training data. Specifically generating tokens that have a lower probability is a surefire way to fool our detector, but at the cost of being significantly more suspect to human readers.
The most likely way our model could be fooled in practice is to embed the secret in as much human written text as possible, since our detector is based on the assumption that a stego text of interest was created by some utilization of a language model. Llama2, as an autoregressive causal model, is unsuited for this approach since it does not support prediction of masked tokens, which is possible with Models of the BERT family that infer token scores from the entire context bidirectionally[4].

[4] https://medium.com/@tom_21755/understanding-causal-llms-masked-llm-s-and-seq2seq-a-guide-to-\protect\penalty\z@language-model-training-d4457bbd07fa, https://huggingface.co/docs/transformers/main/tasks/masked_language_modeling

As Llama2 only predicts a token based on the previous sequence, any approach that would be based on replacing tokens in the text with coded secret information, would require additional effort to blend in the replaced token for enhanced perceptual and semantic imperceptibility. From what we learned throughout this seminar project, we would approach the task of LLM steganography by leveraging a models masking capabilities in the future. This would not only render the hiding process more straight forward and faster, also would the problem of detection have to be reformulated, since the text would contain of mostly human written parts, and lastly cognitive imperceptibility would be preserved.
Another important thought is that more and more cover texts in the coming years might possibly, at least partly, be machine generated which additionally gives reason to question the approach of stego text detection via detection of machine generated texts.

6 Conclusion

In this seminar project we developed a hiding algorithm for generating cover text while at the same time embedding a secret message within, and also built a detection mechanism which was based on the assumption that stego texts are machine generated or altered and which also flags texts that do not match a defined word count.
We found that under the beforementioned assumption, detecting stego texts can be approached by detecting machine generated texts yielding good results. Reversely, we conclude that hiding should involve as little text generation as possible, since good detectors exist and are continuously improved. If generating the cover text cannot be avoided, perceptual imperceptibility can be enhanced by choosing as little low probability tokens as possible during generation.
Hiding messages in texts without making it obvious to neither humans nor automatic detection systems is a hard task. Nevertheless, it seems to be very much possible to at least fool either humans, or automatic systems. Since developing models that achieve a high accuracy, and in particular a low false positive rate, is still an open problem, actors could conceivably hide messages in news articles without too much fear of being caught.
The sheer volume of news articles being published every day combined with the difficulty of maintaining a low false positive rate make it effectively impossible to reliably flag articles that actually contain a hidden message without wasting a lot of manual labor. Since the actual decryption of the messages themselves is a harder task still, and all the technology is already there and mostly usable on consumer hardware, hiding messages in this way is probably already happening. Further research is required in order to reliably detect this.

References

[BZT+24] Guangsheng Bao, Yanbin Zhao, Zhiyang Teng, Linyi Yang, and Yue Zhang. Fast-detectgpt: Efficient zero-shot detection of machine-generated text via conditional probability curvature, 2024.

[MLK+23] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. DetectGPT: Zero-shot machine-generated text detection using probability curvature, 2023.

[NCL18] Shashi Narayan, Shay B. Cohen, and Mirella Lapata. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization, 2018.

[TMS+23] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

[WSZ+24] Yihao Wang, Ruiqi Song, Ru Zhang, Jianyi Liu, and Lingxiao Li. Llsm: Generative linguistic steganography with large language model. *arXiv preprint arXiv:2401.15656, version 3*, 2024.

[YXZ+21] Zhongliang Yang, Lingyun Xiang, Siyu Zhang, Xingming Sun, and Yongfeng Huang. Linguistic generative steganography with enhanced cognitive-imperceptibility. *IEEE Signal Processing Letters*, 28:409–413, 2021.

7 Work split

Alex implemented the original versions of the hider, restorer, and detector, and did the fine-tuning for the detector. He wrote the abstract, most of the detector part, the external evaluation, part of the discussion and conclusion and did minor edits in the hider & restorer part.

Lynn implemented the splitting and reconstructing of the secret into and out of 4-bit and 2-bit chunks, the perturbation for DetectGPT, and did extensive testing and fine-tuning of the hider and restorer to get them to their current performance. She wrote the introduction, most of the hider & restorer part, the internal evaluation, the comparison with FastDetect, parts of the discussion and conclusion and did minor edits in the detector part.

We both believe that these add up to roughly equal amounts of work.